

Superscalar Architecture

Week 2: Dynamic Scheduling — Scoreboarding & Tomasulo's Algorithm

May 22, 2026

Contents

Week 2 in 60 Seconds	4
1 Week 2 — Dynamic Scheduling & Tomasulo’s Algorithm	6
2 Comprehensive Theory 1: Why In-Order Pipelining Isn’t Enough [~20 min]	7
2.1 The Long-Latency Operation Problem	7
2.2 The Compiler Can Only Do So Much	7
2.3 The New Problem: WAR and WAW Become Real	8
3 Comprehensive Theory 2: Scoreboarding (CDC 6600, 1964) [~25 min]	10
3.1 The Scoreboard as a Central Scheduler	10
3.2 What Scoreboarding Achieves	10
3.3 What Scoreboarding Cannot Do	10
4 Comprehensive Theory 3: The Tomasulo Insight [~20 min]	12
4.1 Idea 1: Distributed Scheduling via Reservation Stations	12
4.2 Idea 2: Register Renaming via Tags	12
4.3 Idea 3: The Common Data Bus	13
5 Comprehensive Theory 4: The Algorithm, Step by Step [~35 min]	14
5.1 Stage 1: Issue	14
5.2 Stage 2: Execute	14
5.3 Stage 3: Write-Result	15
5.4 Putting It Together: The Three Tables You Must Draw	15
6 Comprehensive Theory 5: Worked Example — Trace 5 Instructions Cycle by Cycle [~60 min]	17
6.1 The Setup	17
6.2 Cycle 1: Issue I1 (L.D F6)	17
6.3 Cycle 2: Issue I2 (L.D F2). I1 begins executing.	18
6.4 Cycle 3: Issue I3 (MUL F0, F2, F4). Load1 finishes; broadcasts.	18
6.5 Cycle 4: Issue I4 (SUB F8, F6, F2). Load2 finishes; broadcasts.	19
6.6 Cycle 5: Issue I5 (ADD F6, F0, F8). Mult1 and Add1 both begin executing.	19
6.7 Cycle 6: Add1 finishes (SUB).	20
6.8 Cycles 7–14: Mult1 grinds away (10-cycle latency).	20
6.9 Cycle 14: Mult1 finishes.	20
6.10 Cycle 15: Add2 begins executing.	21
6.11 Cycle 16: Add2 finishes.	21
6.12 What to Take Away	21

7	Comprehensive Theory 6: Why Renaming Eliminates WAW and WAR	
	[~15 min]	22
7.1	The Claim	22
7.2	Proof Sketch for WAW	22
7.3	Proof Sketch for WAR	22
7.4	Why This Is So Beautiful	23
8	Comprehensive Theory 7: What Tomasulo Cannot Do — Motivating	
	Week 3 [~15 min]	24
8.1	Problem 1: No Precise Exceptions	24
8.2	Problem 2: No Branch Recovery	24
8.3	Problem 3: No In-Order Commit	24
8.4	The Fix: the Reorder Buffer (ROB)	24
9	Week 2 Exercises — Milestone 1	26
A	Appendix: Solutions to Concept Checks	28

Week 2 in 60 Seconds

A one-page summary. This is the most important week of the entire program — nothing in Weeks 3–8 will work if you don’t internalize what’s below.

The problem Week 2 solves: A 5-stage in-order pipeline (Week 1) stalls whenever any instruction is waiting — even if dozens of *independent* instructions behind it could be executing. We need a way to let later instructions **overtake** stalled ones, while still producing the correct architectural result.

Two historical solutions:

- **Scoreboarding (CDC 6600, 1964):** A central scheduler tracks register usage and lets independent instructions execute out of order. *Problem:* stalls on WAR and WAW hazards, which are now real.
- **Tomasulo’s Algorithm (IBM 360/91, 1967):** Replaces register names with *tags*, making WAR and WAW hazards **disappear automatically**. This is the foundation of every modern out-of-order processor.

The three data structures of Tomasulo:

- **Reservation Stations (RS):** “Waiting rooms” attached to functional units. Each holds one instruction and waits for its operands to arrive on the CDB.
- **Register Status Table (RST):** For each architectural register, says *which reservation station* will produce its next value.
- **Common Data Bus (CDB):** A broadcast bus. When an FU finishes, it puts (tag, value) on the CDB; every RS that’s waiting on that tag captures the value.

The three stages of Tomasulo execution:

1. **Issue:** Take next instruction from queue. Allocate an RS. Look up operands in the RST: if ready, copy values; if not, copy the tag of the producing RS. Update RST so the destination register now points to this RS.
2. **Execute:** Wait until both operand tags are resolved (values captured). Then send to the FU.
3. **Write-Result:** When the FU finishes, broadcast (my-tag, result) on the CDB. Every RS waiting on this tag grabs the value. The register file also writes the result *if* the RST still points to this RS.

The key insight (read three times):

Tags are not register names. Tags are names of reservation stations. The register file is mostly bypassed — values flow from one RS to another via the CDB. A register name is only “correct” at commit time; in between, it’s just a label that points to whichever RS happens to be producing it.

What's still missing (motivates Week 3): Classical Tomasulo has no *precise exceptions* and no *branch recovery*. If a branch is wrong, you can't undo. The Reorder Buffer (Week 3) fixes this.

Suggested reading order: Sections 1–3 build the motivation. Section 4 is the algorithm itself. **Section 5 is the worked example — spend the most time here.** Sections 6–7 cover renaming theory and limitations. Section 8 is the exercises (Milestone 1).

1 Week 2 — Dynamic Scheduling & Tomasulo’s Algorithm

Goal

Internalize Tomasulo’s algorithm at the level required to debug your own Verilog implementation in Week 6. By the end of this week, you should be able to take any short instruction sequence and trace its execution through reservation stations, register status table, and the CDB on paper, cycle by cycle, without notes.

Mentee Work (8–10 hours)

- Read Sections 1–7 of this document carefully. Section 5 (the worked example) deserves at least two passes.
- Watch Onur Mutlu’s lecture on Tomasulo’s algorithm (ETH Computer Architecture, available on YouTube). This is the canonical explanation and complements this document.
- Attempt every Concept Check box as you read. Solutions are in the Appendix.
- Complete **Week 2 Exercises (Section 8)**: a full cycle-by-cycle hand-trace of a 6-instruction loop. **This is Milestone 1.**

Deliverable — Milestone 1

- Push `docs/week2.md` (or PDF) containing:
 - Your cycle-by-cycle Tomasulo trace of the Section 8 exercise, with RS table, RST, and CDB state shown at every cycle.
 - A 1-page write-up answering: *“How does Tomasulo eliminate WAW hazards? Show with a 2-instruction example.”*
- if your trace has tag-tracking errors. Do not start Week 3 Verilog until this is signed off — the bugs will compound.

2 Comprehensive Theory 1: Why In-Order Pipelining Isn't Enough [~20 min]

In Week 1 you built a mental model of a 5-stage pipeline. With forwarding and a single load-use stall, you achieved an IPC very close to 1. So why are we throwing this away?

2.1 The Long-Latency Operation Problem

Consider a sequence with one slow operation (say, a multi-cycle floating-point divide that takes 20 cycles) followed by many independent instructions:

```
div.d    $f0, $f2, $f4      # 20 cycles
add      $s0, $t0, $t1      # 1 cycle, but BLOCKED behind div
sub      $s1, $t2, $t3      # 1 cycle, also blocked
and      $s2, $t4, $t5      # 1 cycle, also blocked
...
```

The `add`, `sub`, and `and` have *no dependence whatsoever* on the divide. They use different registers and a different functional unit (integer ALU instead of FP divider). In a perfect world, they would execute in parallel with the divide, hiding most of its 20-cycle latency.

But our Week 1 pipeline is strictly in-order: it stalls every instruction at the issue stage until the divide commits. We lose 19 cycles of potential work.

You might also think that the second instruction may start in the second cycle, but it will not. If it did, it would finish before the first instruction. This is not a characteristic of an in-order processor. There should be no variable latency. Then both will need to access write-back simultaneously, or the second instruction may finish first. In an in-order execution, the processor is designed so that instructions must enter and leave the pipeline in the exact order they were written in the code. Also, consider whether we are dividing by zero. If we are, and we come to know on cycle 15, an error will be thrown, and the computer's state will be corrupted. The processor cannot safely "undo" those instructions to handle the error.

This is the fundamental limitation of in-order execution. As soon as we add functional units with different latencies (loads from memory, multiplies, divides, FP operations), the pipeline spends most of its time idle, waiting on the slowest operation.

2.2 The Compiler Can Only Do So Much

A smart compiler can reorder instructions at compile time to fill these gaps — this is called *static scheduling*. But the compiler is fundamentally limited:

- It doesn't know cache hit/miss latencies (a load might take 4 cycles or 400).
- It can't reorder across branches without speculation.

- It must produce code that runs on *all* variants of the processor, not just the one with the fastest divider.

We need the hardware itself to dynamically reorder instructions *at runtime*, based on the actual readiness of operands and functional units. This is called **dynamic scheduling**.

2.3 The New Problem: WAR and WAW Become Real

Week 1 dismissed WAR (Write-After-Read) and WAW (Write-After-Write) as impossible in an in-order pipeline. They are. But they reappear instantly the moment we allow instructions to execute out of order.

```
div.d    $f0, $f2, $f4      # Writes $f0 (takes 20 cycles)
add      $s0, $t0, $t1      # Reads $t0
mul      $f0, $f6, $f8      # ALSO writes $f0 (1 cycle)
```

If we let `mul` execute out of order while `div.d` is still computing, the `mul` finishes first and writes its result to `$f0`. Twenty cycles later, the `div.d` finishes and overwrites `$f0` with the *wrong* (now-stale) value. This is a WAW hazard, and it would silently corrupt program state.

WAR is similar: if a later instruction writes to a register that an earlier, still-pending instruction is about to read, we must ensure the read sees the old value, not the new one.

If you think to do this through in-order execution: In the first two cycles, the `div.d` instruction enters the pipeline and moves into its lengthy execution phase. By cycle 3, as the `add` instruction reaches the Decode stage and `mul` is Fetched, the strictly in-order hardware detects that `div.d` will take 20 cycles and immediately stalls `add` to maintain sequential order. This creates a massive bottleneck from cycles 4 through 22, where the pipeline sits mostly idle—`add` remains trapped in Decode, `mul` is stuck in Fetch, and the integer units do absolutely zero work. It isn't until cycles 23 and 24 that `div.d` finally finishes calculating and safely writes its result to `$f0` during its Write Back stage. With the register now safely updated, the pipeline unfreezes in cycle 25, finally allowing `mul` to decode, execute, and eventually write its own result to `$f0` without causing a hazard.

So before we even start designing an out-of-order machine, we have a problem to solve: how do we prevent WAW and WAR hazards from corrupting state? The answer turns out to be deeply elegant, and we'll get to it in Section 4.

Concept Check 1

1. Why is a long-latency divide especially harmful in an in-order pipeline, while a long-latency *cache miss* for a load is only mildly harmful in the same pipeline? (Hint: think about which stage stalls.)
2. Give an example of a WAR hazard in a 3-instruction sequence where the second instruction reads a register that the third writes.
3. Why can't a static compiler always reorder code to eliminate the long-divide stall?

Going deeper: Hennessy & Patterson, Quantitative Approach, §3.4 introduction provides additional motivation. Onur Mutlu's lectures on out-of-order execution begin with this same problem.

3 Comprehensive Theory 2: Scoreboarding (CDC 6600, 1964) [~25 min]

The first commercial computer to attempt dynamic scheduling was the CDC 6600 in 1964, designed by Seymour Cray. Its mechanism is called **scoreboarding**. We study it not because anyone uses it today, but because it teaches us exactly which problems Tomasulo’s algorithm later solves.

3.1 The Scoreboard as a Central Scheduler

A **scoreboard** is a central piece of hardware that tracks:

- Which functional units are busy.
- Which architectural registers are about to be written (and by which FU).
- Which instructions are waiting for operands.

When the front-end fetches an instruction, the scoreboard checks four conditions before allowing it to proceed:

1. **Issue:** Is there a free functional unit? Is the destination register *not* about to be written by another in-flight instruction (otherwise WAW)? If yes to both, issue. Otherwise stall.
2. **Read operands:** Are both source operands available in the register file? If yes, send them to the FU and start execution. If not, wait.
3. **Execute:** The FU computes the result.
4. **Write result:** Before writing back, check: is any earlier instruction still reading the destination register? If yes (WAR), stall. If no, write.

3.2 What Scoreboarding Achieves

Independent instructions can now overtake each other. In our earlier divide-then-add example, the **add** no longer stalls behind the divide — it just needs to find a free integer ALU, and it can proceed.

3.3 What Scoreboarding Cannot Do

Scoreboarding has three fatal weaknesses, and understanding them is the whole point of studying it:

1. **WAW stalls at issue.** If two instructions both write **\$f0**, the scoreboard refuses to issue the second one until the first completes. Pipeline stalls for the entire latency of the first instruction.

2. **WAR stalls at write-back.** If a later instruction wants to write a register that an earlier instruction is still reading, the writer waits. Again, a forced stall.
3. **No forwarding between FUs.** When an FU finishes, it writes to the register file. The next instruction that needs that value must wait for the write to complete, then read the register file. No direct producer-to-consumer bypass.

These three weaknesses share a common root cause: **the scoreboard tracks register names, not data flow**. When two instructions both write `$f0`, the scoreboard sees a conflict over the name `$f0`, even though there is no actual conflict (this means that there is no problem in the execution unit but sure there may be a race between who gets to write-back, `mul` or `add`. The final value taken by `$f0` now decides the exact sequence the instructions were written in by the programmer or compiler. (uses ROB, a special hardware structure)) over the *data* being computed. The two writes are simply two independent values that happen to share a label.

Tomasulo's central insight was: *stop tracking register names, and start tracking the data itself*.

Concept Check 2

1. In a scoreboard, an instruction is allowed to start executing only when both its source operands are in the register file. Why is this slower than forwarding the operands directly from a producing FU's output?
2. Suppose a scoreboarded machine encounters: `add $f0, $f2, $f4` followed immediately by `mul $f0, $f6, $f8`. How does the scoreboard prevent the WAW hazard, and what is the performance cost?

Going deeper: Hennessy & Patterson, Quantitative Approach, §C.7 (online appendix) covers scoreboarding in full depth. For this course you only need the conceptual understanding above — we will not implement scoreboarding.

4 Comprehensive Theory 3: The Tomasulo Insight

[~20 min]

In 1967, Robert Tomasulo published an algorithm for the IBM System/360 Model 91 that solved every weakness of the scoreboard simultaneously. The algorithm has remained the conceptual foundation of every high-performance processor for nearly sixty years. What follows is a description of his three key ideas.

4.1 Idea 1: Distributed Scheduling via Reservation Stations

Instead of one central scoreboard, every functional unit (this can be integer alu, multiplier, floating point unit, load/store unit) gets its own small queue of waiting instructions (earlier it was for whole execution unit, now it for individual FU). These queues are called **reservation stations** (RS).

When an instruction is issued, it does not wait at a central scheduler. It is immediately sent into a reservation station attached to the FU it will use (integer ALU, multiplier, load/store unit, etc.). Each RS holds one instruction, plus its operands (or placeholders for them if they're not ready yet).

Consequence: The processor can have many more in-flight instructions than the scoreboard allowed, because each FU has multiple RS slots in parallel.

4.2 Idea 2: Register Renaming via Tags

Here is the deepest idea in this entire course. Read it slowly.

In a scoreboard, when an instruction asks “what is the value of `$f0`?”, the scheduler looks at the register file. If `$f0` is currently being written by another instruction, we have a hazard.

In Tomasulo, that question is never asked. Instead, the instruction asks: “*which reservation station is currently producing `$f0`?*” The answer is either a tag (e.g., “RS#3 is producing `$f0`”) or, if no in-flight instruction is producing it, the value from the register file.

The **Register Status Table (RST)** stores exactly this mapping: for every architectural register, it remembers which RS is going to produce its next value. When a new instruction is issued that writes `$f0`, the RST is updated: “from now on, `$f0` is produced by RS#7” (the RS we just allocated for the new instruction).

What happens to the previous mapping (“`$f0` was produced by RS#3”)? It is **forgotten**. The instruction in RS#3 will still execute and produce a value, but no one is listening anymore. Its value will be written to the register file, but immediately overwritten by RS#7's later value — which is exactly what we wanted, because `$f0`'s “true” value (in program order) is the one from RS#7.

WAW dissolves. The two writes happen in whatever order their FUs finish, but the RST always points to the *latest* producer in program order. WAW is no longer a hazard; it is a non-event.

WAR dissolves too. If a later instruction wants to write `$f0` while an earlier instruction is still computing something that reads `$f0`, the earlier instruction *already captured the old tag* at issue time. It will receive the old value when that RS completes. The later instruction’s write to `$f0` (via its own RS) cannot affect the earlier one, because they are tracking different tags.

This is the renaming insight. Architectural register names are just labels; the actual data flow is described by tags. Two instructions that “write the same register” are actually producing two completely independent values, just with the same name.

4.3 Idea 3: The Common Data Bus

When an FU finishes executing, it has a result that needs to go to:

- Every RS that’s waiting on this result as an operand.
- The register file (if this RS is still the “current producer” for some architectural register).

In Tomasulo, this is done with a single broadcast bus called the **Common Data Bus (CDB)**. The completing FU drives (**my-tag**, **my-value**) onto the CDB. Every RS in the machine snoops the CDB: if either of its two operand tags matches the broadcasted tag, it captures the value and marks that operand ready. The register file does the same: if its RST entry for any register still points to this tag, it captures the value.

Consider this as a CPU-wide internal broadcast network. If the subsequent instruction needs data, it doesn’t need to wait till writeback. After finishing the work from the FU, the result is broadcast across the entire processor in the form (my-tag, my-value). Let’s say the tag is RS#5; the RS slot waiting for this (this makes data forwarding free here, as there is direct producer to consumer contact) also starts working now, since it has the value. Also, since the RS#5 is completed, the register file updates itself.

Concept Check 3

1. In your own words, explain why tags eliminate WAW hazards. Use a 2-instruction example.
2. The CDB is a single shared bus. What happens if two FUs finish in the same cycle? (You may not know the full answer yet — guess based on first principles.)
3. Suppose an instruction in RS#3 is waiting for an operand that “belongs to RS#5.” Halfway through RS#5’s execution, a *new* instruction is issued that also writes the same architectural register, going into RS#9. Does RS#3 now wait for RS#5 or RS#9? Why?

Going deeper: Tomasulo’s original 1967 paper, “An Efficient Algorithm for Exploiting Multiple Arithmetic Units,” is short and readable. Find it on IBM’s archives or via Google Scholar.

5 Comprehensive Theory 4: The Algorithm, Step by Step [~35 min]

Now we make the three insights from Section 3 concrete. Every Tomasulo execution proceeds through three stages: **Issue**, **Execute**, **Write-Result**.

5.1 Stage 1: Issue

The front-end has decoded the next instruction in program order. The Issue stage attempts to place it into an appropriate reservation station.

Step-by-step:

1. **Pick an RS.** Look at the FU type the instruction needs (ALU, multiplier, load-store). Pick a free RS attached to that FU. *If no RS is free, stall the front-end — this is called a structural hazard on RS slots, and it's the main reason real processors have many RSs.*
2. **Resolve operand 1 (and operand 2 similarly).** Look up the source register in the Register Status Table.
 - If the RST says “no in-flight producer” for this register, read the value directly from the register file. Store the value in the RS's operand-1-value field. Set operand-1-tag to “ready” (often represented as 0 or a special null tag).
 - If the RST says “producer is RS#k”, do *not* read the register file. Instead, copy the tag k into the RS's operand-1-tag field. The value will arrive later via the CDB.
3. **Update the RST.** The instruction's destination register now has a new producer: this RS. Overwrite the RST entry for the destination register with this RS's tag.

After Issue, the RS contains: the operation, two operand slots (each either “value ready” or “waiting for tag k”), and metadata. The instruction now waits for both operands to become ready.

5.2 Stage 2: Execute

This stage has two sub-steps: *wait for ready*, then *compute*.

Wait for ready. Every cycle, the RS checks: are both operand tags resolved (i.e., do both operand-value fields hold real values)? This happens automatically through CDB snooping (see Stage 3). If yes, the RS is ready to fire.

Select and compute. Among all ready RSs attached to the same FU, only one can fire per cycle (the FU is a serial resource). A simple priority-encoder picks one. The selected RS sends its operation and both operand values to the FU, which begins computing. The RS itself remains allocated — it cannot be reused until the result is written.

For multi-cycle FUs (multiplier, divider), the RS is occupied for the full latency of the operation.

5.3 Stage 3: Write-Result

When the FU finishes computing, the result needs to be broadcast.

Step-by-step:

1. **Drive the CDB.** The FU (on behalf of the RS that issued it) places (**my-tag**, **my-value**) on the CDB. If multiple FUs finish in the same cycle, arbitration logic picks one (the others must stall their result one cycle).
2. **Every RS snoops.** Every reservation station in the machine compares its two operand tags against the CDB's broadcast tag. If either matches: capture the value into the operand-value field, mark that operand ready, and clear the operand tag.
3. **Register file snoops too.** For each architectural register, check the RST: is its current producer-tag equal to the broadcast tag? If yes: write the broadcast value into the register file AND clear the RST entry (back to “no in-flight producer”). If no: do nothing — a later instruction has already claimed this register, so this value is architecturally dead.
4. **Free the RS.** The reservation station is now empty and available for the next Issue.

That third sub-step is subtle and deeply important: it's how Tomasulo automatically discards the results of “shadowed” computations. If two instructions both write `$f0`, only the one whose tag is currently in the RST will actually update the register file. The other one broadcasts on the CDB (so any RS waiting on its tag still gets its value), but the register file ignores it.

5.4 Putting It Together: The Three Tables You Must Draw

A Tomasulo state at any given cycle consists of three tables:

1. **Reservation Station Table.** One row per RS. Columns:

- **Busy:** Is this RS occupied?
- **Op:** What operation (ADD, MUL, LD, etc.)?
- **V_j , V_k :** Operand values (if ready).
- **Q_j , Q_k :** Operand tags (if not ready; null/0 if value already captured).
- **Dest** or **A** (load): destination info or address.

2. Register Status Table. One row per architectural register. One column:

- Q_i : tag of the RS currently producing this register, or null if no in-flight producer.

3. CDB state. What's on the bus this cycle: either nothing, or (tag, value).

Drawing these three tables at every cycle is the entire skill of Week 2. The exercise in Section 8 is exactly this. Practice now with the worked example in Section 5.

Concept Check 4

1. During Issue, if a source register's RST entry says "producer is RS#5," we copy the tag, not the value. Why is reading the register file at this point definitely the *wrong* thing to do?
2. Why must we update the RST entry for the destination register *during* Issue, not later?
3. During Write-Result, the register file might choose not to capture the broadcast value (Step 3 above). Walk through a specific scenario where this happens.

Going deeper: Hennessy & Patterson, Quantitative Approach, §3.4 and §3.5 cover the algorithm in formal detail. Read §3.5 alongside this section.

6 Comprehensive Theory 5: Worked Example — Trace 5 Instructions Cycle by Cycle [~60 min]

This is the most important section of Week 2. Do not just read it — copy out the tables on paper and re-draw them as you follow along. If you cannot reproduce this trace from scratch by the end of the week, you are not ready for Milestone 1.

6.1 The Setup

Code:

```
I1:  L.D      F6, 0(R2)          # Load from memory
I2:  L.D      F2, 0(R3)          # Load from memory
I3:  MUL.D    F0, F2, F4          # F0 = F2 * F4      (depends on I2)
I4:  SUB.D    F8, F6, F2          # F8 = F6 - F2      (depends on I1 and I2)
I5:  ADD.D    F6, F0, F8          # F6 = F0 + F8      (depends on I3 and I4; also
```

Hardware:

- 2 Load reservation stations (Load1, Load2).
- 2 Add/Subtract reservation stations (Add1, Add2).
- 2 Multiply reservation stations (Mult1, Mult2).
- Latencies: Load = 2 cycles, Add/Sub = 2 cycles, Mul = 10 cycles.
- One CDB. One issue per cycle (single-issue Tomasulo — not yet superscalar).

Initial state: all RSs empty; RST entries all null; F4, R2, R3 hold known register-file values (we don't care what they are for this trace).

6.2 Cycle 1: Issue I1 (L.D F6)

Issue picks Load1 for I1. R2 has no in-flight producer (RST[R2] = null), so its value is read from the register file. F6's RST entry is updated to "Load1."

Resulting state:

RS	Busy	Op	V_j	V_k	Q_j	Q_k
Load1	yes	LD	[R2]	—	0	—
Load2	no					
Add1	no					
Add2	no					
Mult1	no					
Mult2	no					

RST: F6 $\rightarrow$ Load1. All others null.

CDB: nothing.

6.3 Cycle 2: Issue I2 (L.D F2). I1 begins executing.

I2 issues to Load2. $\text{RST}[\text{F2}] \rightarrow \text{Load2}$. Meanwhile, Load1 has both operands ready ($V_j = \text{R2's value}$; loads have no second operand), so it begins executing this cycle. It will take 2 cycles — it'll be done at the end of cycle 3.

Resulting state:

RS	Busy	Op	V_j	V_k	Q_j	Q_k
Load1	yes (executing)	LD	[R2]	—	0	—
Load2	yes	LD	[R3]	—	0	—
<i>(other RSs empty)</i>						

RST: $\text{F6} \rightarrow \text{Load1}$, $\text{F2} \rightarrow \text{Load2}$.

CDB: nothing.

6.4 Cycle 3: Issue I3 (MUL F0, F2, F4). Load1 finishes; broadcasts.

I3 issues to Mult1. Looking up its operands:

- F2: $\text{RST}[\text{F2}] = \text{Load2}$ (not ready). Copy tag Load2 into Mult1's Q_j .
- F4: $\text{RST}[\text{F4}] = \text{null}$. Read F4's value from the register file into V_k .

$\text{RST}[\text{F0}] \rightarrow \text{Mult1}$.

Load1 finishes computing (it started in cycle 2 with 2-cycle latency, so it's done at end of cycle 3). **It places (Load1, mem[R2]) on the CDB.**

What snoops the CDB?

- No RS is waiting on tag Load1 right now (Mult1 wants Load2, not Load1).
- $\text{RST}[\text{F6}] = \text{Load1}$. So the register file captures $\text{mem}[\text{R2}]$ into F6 and clears $\text{RST}[\text{F6}]$ to null.

Load1 is now freed.

Resulting state:

RS	Busy	Op	V_j	V_k	Q_j	Q_k
Load1	no					
Load2	yes (executing)	LD	[R3]	—	0	—
Mult1	yes	MUL	—	[F4]	Load2	0

RST: $\text{F2} \rightarrow \text{Load2}$, $\text{F0} \rightarrow \text{Mult1}$. F6 just got its value — $\text{RST}[\text{F6}]$ is null.

CDB: (Load1, $\text{mem}[\text{R2}]$).

Notice: F6 now holds I1's value. But I5 will overwrite F6 later. That's fine — the RST will be updated again when I5 issues, and the old F6 value will simply be replaced.

6.5 Cycle 4: Issue I4 (SUB F8, F6, F2). Load2 finishes; broadcasts.

I4 issues to Add1. Operands:

- F6: RST[F6] = null (the load just completed). Read F6's value (mem[R2]) from the register file into V_j .
- F2: RST[F2] = Load2 (not ready). Copy tag Load2 into Add1's Q_k .

RST[F8] $\rightarrow$ Add1.

Load2 finishes (started cycle 3, 2-cycle latency, done end of cycle 4). **Broadcasts** (Load2, mem[R3]) **on the CDB**.

CDB snooping:

- Mult1's Q_j = Load2 $\rightarrow$ match! Mult1 captures mem[R3] into V_j and clears Q_j . Both Mult1's operands are now ready.
- Add1's Q_k = Load2 $\rightarrow$ match! Add1 captures mem[R3] into V_k and clears Q_k . Both Add1's operands are now ready.
- RST[F2] = Load2 $\rightarrow$ register file captures mem[R3] into F2 and clears RST[F2].

A single CDB broadcast just delivered the value to two waiting consumers AND updated the register file. This is the magic of Tomasulo.

Resulting state:

RS	Busy	Op	V_j	V_k	Q_j	Q_k
Load2	no					
Add1	yes (ready)	SUB	[F6]	[F2]	0	0
Mult1	yes (ready)	MUL	[F2]	[F4]	0	0

RST: F0 $\rightarrow$ Mult1, F8 $\rightarrow$ Add1.

CDB: (Load2, mem[R3]).

6.6 Cycle 5: Issue I5 (ADD F6, F0, F8). Mult1 and Add1 both begin executing.

I5 issues to Add2. Operands:

- F0: RST[F0] = Mult1 (not ready, executing). Copy Mult1 into Add2's Q_j .
- F8: RST[F8] = Add1 (not ready, executing). Copy Add1 into Add2's Q_k .

RST[F6] $\rightarrow$ Add2. This is where the WAW resolves automatically: F6 used to point to Load1, then briefly held the value of I1, and now points to Add2. The old value from I1 is simply overwritten by whatever I5 produces. No stall needed.

Mult1 and Add1 both have all operands ready. They begin executing. Mult1 takes 10 cycles (done end of cycle 14). Add1 takes 2 cycles (done end of cycle 6).

Resulting state:

RS	Busy	Op	V_j	V_k	Q_j	Q_k
Add1	yes (executing)	SUB	[F6]	[F2]	0	0
Mult1	yes (executing)	MUL	[F2]	[F4]	0	0
Add2	yes	ADD	—	—	Mult1	Add1

RST: F0 $\rightarrow$ Mult1, F8 $\rightarrow$ Add1, F6 $\rightarrow$ Add2.

CDB: nothing.

All five instructions have now issued. Let's fast-forward through execution.

6.7 Cycle 6: Add1 finishes (SUB).

Add1 broadcasts (Add1, result_of_sub) on the CDB.

- Add2's $Q_k = \text{Add1} \rightarrow$ match! Captures the SUB result into V_k , clears Q_k . Add2 now waits only on Mult1.
- RST[F8] = Add1 $\rightarrow$ register file writes F8 and clears RST[F8].

Add1 is freed.

6.8 Cycles 7–14: Mult1 grinds away (10-cycle latency).

Nothing happens at the algorithm level for these cycles. Mult1 continues computing. Add2 sits waiting on Mult1. The CDB is idle.

6.9 Cycle 14: Mult1 finishes.

Mult1 broadcasts (Mult1, mul_result).

- Add2's $Q_j = \text{Mult1} \rightarrow$ match! Captures mul_result into V_j , clears Q_j . Add2 is now ready.
- RST[F0] = Mult1 $\rightarrow$ register file writes F0, clears RST[F0].

Mult1 is freed.

6.10 Cycle 15: Add2 begins executing.

Add2 had both operands as of end of cycle 14. It begins executing now, will take 2 cycles, done end of cycle 16.

6.11 Cycle 16: Add2 finishes.

Add2 broadcasts (Add2, final_F6_value).

- No RS waiting on Add2.
- $RST[F6] = \text{Add2} \rightarrow$ register file writes F6, clears $RST[F6]$.

Done. Total: 16 cycles for 5 instructions. The 10-cycle multiply was almost fully overlapped with other work — the SUB, all the issues, and three loads happened during it. This is the IPC win from out-of-order.

6.12 What to Take Away

- I3 (multiply) issued in cycle 3 but didn't begin executing until cycle 5 — it had to wait for I2 to provide F2. During that wait, I4 and I5 issued behind it. **The pipeline did not stall.**
- I5 wrote F6, the same register I1 wrote. WAW was handled silently by the RST update in cycle 5. **No special hazard logic needed.**
- The CDB broadcast in cycle 4 delivered Load2's value to two consumers and the register file in a single action. **Forwarding is free.**
- At cycle 14, when Mult1 finally finishes, Add2 (waiting since cycle 5) instantly picks up its operand. The 9-cycle wait was not a stall — the pipeline kept doing other things.

Concept Check 5

1. In cycle 5, when I5 (ADD F6, F0, F8) issues, why don't we just write its operand values directly from the register file — which already holds $F6 = \text{mem}[R2]$?
2. In cycle 3, the CDB broadcast (Load1, $\text{mem}[R2]$) found no RS waiting for it. Was the broadcast wasted, or did it accomplish something?
3. Suppose Mult1 took only 1 cycle instead of 10. Trace what would change in cycles 5–14. (You don't have to write the full trace, just describe the differences.)

7 Comprehensive Theory 6: Why Renaming Eliminates WAW and WAR [~15 min]

We have asserted this several times. Let us now prove it rigorously.

7.1 The Claim

In Tomasulo, two instructions that write the same architectural register never need to stall on each other, and never need to wait for ordering with respect to earlier readers.

7.2 Proof Sketch for WAW

Suppose I_a and I_b are two instructions in program order ($a < b$) that both write to register R. We want to show that the architecturally-correct final value of R is I_b 's result (the later writer “wins”), regardless of which RS finishes first.

At the time I_a issues: $RST[R]$ is updated to “produced by RS_a .”

At the time I_b issues (which happens after I_a 's issue, since issue is in-order): $RST[R]$ is overwritten to “produced by RS_b .”

When RS_a finishes: It broadcasts (RS_a , $value_a$). The register file checks $RST[R]$: it equals RS_b , not RS_a . **So the register file ignores this broadcast.** $value_a$ is delivered to any RS that captured tag RS_a (i.e., instructions between I_a and I_b that read R), but it does NOT update the register file.

When RS_b finishes: It broadcasts (RS_b , $value_b$). $RST[R] = RS_b$, so the register file captures $value_b$ into R and clears $RST[R]$.

Result: R holds $value_b$. The order in which RS_a and RS_b finished did not matter. WAW dissolved.

7.3 Proof Sketch for WAR

Suppose I_a reads R, and I_b ($b > a$) writes R. We want to show that I_a always reads the architecturally-correct version of R (the value of R *before* I_b 's write).

At the time I_a issues: I_a reads $RST[R]$. Either it finds null (no in-flight producer) and reads R's current value from the register file, or it finds some tag T (an even-earlier producer) and copies tag T. Either way, I_a has *committed to a specific source* for R at this moment.

At the time I_b issues (after I_a): $RST[R]$ is updated to RS_b . But this update does not retroactively change anything I_a has captured. I_a 's reservation station already holds either R's old value or the old producer tag T.

When RS_b finishes: It broadcasts (RS_b , $value_b$). I_a 's reservation station checks: “am I waiting on tag RS_b ?” The answer is no — I_a is either waiting on T or already has the value. **So I_a ignores RS_b 's broadcast.**

Result: I_a eventually executes using R's old value, exactly as program order requires. WAR dissolved.

7.4 Why This Is So Beautiful

The hardware does not contain any explicit “WAW logic” or “WAR logic.” There is no comparator that checks for these hazards. They simply cannot occur, because the design’s invariants prevent them from being expressible.

This is what computer architects mean when they say a design is *elegant*. Tomasulo did not solve WAW and WAR; he made them impossible.

Concept Check 6

1. In the WAR proof above, what if I_a ’s reservation station happens to have the same tag value as RS_b by coincidence (e.g., both are called “RS#5” at different times)? Why isn’t this a problem in practice?

8 Comprehensive Theory 7: What Tomasulo Cannot Do — Motivating Week 3 [~15 min]

Classical Tomasulo (1967) is brilliant but incomplete. Three things it cannot do are the entire reason Week 3 exists.

8.1 Problem 1: No Precise Exceptions

Suppose I_a is an add that, by an unlucky combination of inputs, raises a floating-point overflow exception. Meanwhile, I_b (a later instruction) has already finished executing and written to the register file via the CDB.

When the exception is signaled, the architectural state is wrong: I_b 's effects are already visible, even though I_b should not have executed (because I_a , which precedes it in program order, faulted). The OS handler cannot inspect a clean “state at the moment of the fault.”

This is called the *imprecise exception* problem, and it makes classical Tomasulo unusable for any system that needs reliable exception handling — which is, in practice, every modern system.

8.2 Problem 2: No Branch Recovery

Suppose we speculate that a branch is not taken, and fetch & issue 10 instructions past it. Five of them complete and write the register file. Then the branch resolves: it *was* taken; we should not have executed any of those 10 instructions.

Tomasulo has no mechanism to undo this. The committed writes have already destroyed the previous register values.

8.3 Problem 3: No In-Order Commit

Even ignoring exceptions and branches, debugging and reasoning about a Tomasulo machine is painful because writes happen in execution order, not program order. A user who asks “what is the value of F0 right now?” may get a different answer depending on which RS happened to finish last.

8.4 The Fix: the Reorder Buffer (ROB)

All three problems share a single solution. We add a structure called the **Reorder Buffer** that:

- Allocates an entry for every issued instruction, in program order.
- Holds the instruction's result *privately* until it reaches the head of the ROB.
- Only at that point “commits” the result to the architectural register file.

- If anything goes wrong before commit (exception, branch mispredict), simply flush all ROB entries from the offending instruction onward. Architectural state is untouched.

The ROB sits between the CDB and the architectural register file. The CDB still broadcasts to waiting RSs (so forwarding remains fast), but the architectural “commit” step is decoupled and happens strictly in program order.

This is what you will build in Week 3. The ROB is the structure that turns classical Tomasulo into a modern out-of-order processor. It is also where “register renaming” shifts from being tag-based (Tomasulo’s classical view) to physical-register-file-based (BOOM’s view) — and we’ll see in Week 3 that these are conceptually identical, just implemented differently.

Concept Check 7

1. Why does writing to the register file “out of program order” make exceptions imprecise? Walk through an example.
2. The ROB delays commit until in-order. Doesn’t this just re-introduce the in-order-pipeline problem we tried to escape from? (Hint: consider what happens at *commit* vs. what happens at *execute*.)

Going deeper: Smith & Pleszkun’s 1985 paper, “Implementation of Precise Interrupts in Pipelined Processors,” introduced the ROB. It is one of the most-cited papers in computer architecture.

9 Week 2 Exercises — Milestone 1

This is the gate of the entire program. **If you cannot do this trace correctly, you will not be able to debug your Verilog ROB in Week 6.** Submit your answers in docs/week2.md (or PDF).

Part A — Cycle-by-Cycle Trace

Code:

```
I1:  L.D      F2, 0(R1)           # Load
I2:  MUL.D    F4, F2, F6          # F4 = F2 * F6 (depends on I1)
I3:  ADD.D    F8, F2, F10         # F8 = F2 + F10 (depends on I1)
I4:  SUB.D    F4, F8, F2          # F4 = F8 - F2 (WAW on F4 with I2; depends
I5:  L.D      F8, 8(R1)          # Load (WAW on F8 with I3)
I6:  ADD.D    F12, F4, F8         # F12 = F4 + F8 (depends on I4 and I5)
```

Hardware:

- 2 Load reservation stations, 2 Add/Sub RSs, 2 Multiply RSs.
- Latencies: Load = 2 cycles, Add/Sub = 2 cycles, MUL = 6 cycles.
- Single-issue (one issue per cycle), single CDB.

What to draw, every cycle, until all 6 instructions complete:

1. The Reservation Station table (Busy, Op, V_j , V_k , Q_j , Q_k).
2. The Register Status Table (which RS is producing each register in F2, F4, F8, F12).
3. What's on the CDB this cycle (or “nothing”).
4. A short note for that cycle: “I3 issued to Add1,” “Load2 finished, broadcast captured by Mult1 and Add1,” etc.

You can use a spreadsheet, a markdown table, or hand-drawn scans. What matters is that the trace is *correct* and *readable*.

Part B — The WAW Question

In a 1-page write-up (no more, no less), answer:

“Show, with a 2-instruction example, exactly how Tomasulo’s algorithm prevents a WAW hazard from corrupting the architectural register file. Be specific about which fields of the Register Status Table and reservation stations change, and at which cycles.”

You may use one of the WAW pairs from Part A’s exercise as your example (I2 & I4, or I3 & I5), or invent your own.

Grading Rubric (for your own reference)

- **Trace correctness (60%):** Tag tracking exactly right; CDB broadcasts go to the correct snoopers; RST updates happen at issue time.
- **Trace completeness (20%):** Every cycle shown until all instructions commit.
- **WAW write-up (20%):** Mechanism clearly explained, with specific table states.

A Appendix: Solutions to Concept Checks

Try every question on your own first. Looking up answers prematurely will hurt you at Milestone 1.

Concept Check 1 (Why In-Order Isn't Enough)

1. A long divide stalls at the *issue* or *execute* stage, blocking everything behind it from even starting. A cache miss for a load stalls only the dependent instructions waiting on the load's result; truly independent later instructions are not blocked — they can continue (in a machine that has any kind of forwarding/scoreboarding). The blocking point matters: the earlier in the pipeline you stall, the more instructions are penalized.
2. Example: `add $s0, $t0, $t1; or $s2, $s3, $s4; sub $s3, $s5, $s6`. The `or` reads `$s3`; the `sub` writes `$s3`. If we let the `sub` execute before the `or` finishes reading, the `or` gets the wrong value. This is WAR.
3. The compiler doesn't know cache hit/miss latency at compile time, can't reorder across many branches, and must produce code that works on all variants of the processor (some of which might have a fast divider, some slow). Static scheduling has fundamental limits.

Concept Check 2 (Scoreboarding)

1. Because the value has to go: producer FU → register file (one cycle) → register file read → consumer FU. That's two extra cycles compared to a direct producer-to-consumer forwarding path. With forwarding (which Tomasulo will introduce via the CDB), the producer-to-consumer is one cycle.
2. The scoreboard refuses to *issue* the `mul` until the `add` completes (because both write `F0`, which is a WAW conflict at issue time). The pipeline stalls for the full latency of the `add` — e.g., 2 cycles. The independent `mul` cannot proceed even though there's no real data conflict between them.

Concept Check 3 (The Tomasulo Insight)

1. **In your own words, explain why tags eliminate WAW hazards. Use a 2-instruction example.**

Tags eliminate Write-After-Write (WAW) hazards because Tomasulo's algorithm shifts the hardware's focus from tracking static register names to tracking the dynamic physical producer of the data.

Consider the sequence:

```

add $f0, $f2, $f4 # Allocated to RS#1
mul $f0, $f6, $f8 # Allocated to RS#2

```

Both instructions target the same architectural destination register, `$f0`. At issue time, the Register Status Table (RST) entry for `F0` is overwritten sequentially: first it points to the `add`'s reservation station (`RS#1`), then immediately to the `mul`'s reservation station (`RS#2`) because it is the younger instruction in program order.

If the `mul` instruction executes rapidly out-of-order and finishes first, it broadcasts its tag (`RS#2`) and its product onto the Common Data Bus (CDB). The register file checks `RST[F0]`, sees a match, and captures the value. When the slow `add` eventually finishes and broadcasts later, the register file checks `RST[F0]` again. Because it now points to the `mul`'s RS (`RS#2`) and not the `add`'s (`RS#1`), the `add`'s broadcast is safely ignored for register-file updates. The `mul`'s later broadcast wins, and the WAW hazard is silently resolved without requiring any pipeline stalls.

2. The CDB is a single shared bus. What happens if two FUs finish in the same cycle?

Because the CDB is a single physical wire network, only one functional unit can broadcast at a time. If two FUs finish executing in the exact same cycle, a structural conflict occurs and one must wait.

Typically, a hardware bus arbiter intervenes to resolve the collision. The FU that arbitrates and wins the CDB gets to broadcast its (`tag`, `value`) pair during this cycle. The losing FU is forced to stall its write-back phase, holding its completed result internally for one more cycle before trying to arbitrate and broadcast again on the next cycle. We'll formalize this arbitration logic in Week 4 / Week 6.

3. *Suppose an instruction in RS#3 is waiting for an operand that “belongs to RS#5.” Halfway through RS#5's execution, a new instruction is issued that also writes the same architectural register, going into RS#9. Does RS#3 now wait for RS#5 or RS#9? Why?

`RS#3` continues to wait for `RS#5`.

The Q_j/Q_k operand source fields of `RS#3` are explicitly set *at issue time*. At that exact moment, the instruction captured tag `RS#5` because it was the current valid producer listed in the RST. Later changes to the RST (which overwrite the entry to say “`$f0` is now produced by `RS#9`”) only affect future instructions entering the pipeline; they do not retrospectively alter `RS#3`'s already-captured operand tags. `RS#3` will remain completely oblivious to `RS#9`, listening to the CDB specifically for the `RS#5` broadcast. This strict binding at issue time is exactly how Write-After-Read (WAR) hazards are prevented.

Concept Check 4 (The Algorithm in Detail)

1. Because the register file currently holds a *stale* value — the previous version of the register, before the in-flight producer’s update. Reading it would give the wrong value. We must wait for the producing RS to deliver the correct value via the CDB.
2. Because the next instruction in program order might read the same register — and we want it to see this instruction as the producer (not whoever was there before). If we delayed the update, the next instruction might pick up the stale producer tag, breaking program order.
3. Scenario: I1 writes F0 (allocated to RS#3). I2 (later in program order) also writes F0 (allocated to RS#7). When I1’s RS#3 broadcasts its result, the register file checks RST[F0]: it equals RS#7, not RS#3. So the register file ignores the broadcast for itself, but the RS that captured RS#3 as a tag (if any) still receives the value.

Concept Check 5 (Worked Example)

1. Because F6’s value in the register file is *stale* — it holds I1’s value, but the program order says I5 should see the value of F6 *after* I4 might have modified it. Wait: actually, I4 doesn’t write F6, it writes F8. And I5 doesn’t read F6 as a source! Re-read I5: “ADD F6, F0, F8.” F6 is the *destination*, not a source. So this question is a trick — we never need F6’s value to issue I5. We only need F0 (waiting on Mult1) and F8 (waiting on Add1). The RST values for those tell us correctly.
2. Not wasted. The register file captured mem[R2] into F6. RST[F6] cleared to null. This was the architectural commit of I1’s result. The broadcast is also a no-op for RSs because no later instruction (yet) is waiting on Load1 — they’re all waiting on Load2, since F2 was the dependency they cared about.
3. If Mult1 took 1 cycle: it would finish at end of cycle 5 (instead of cycle 14). Add2 would get its operand from Mult1’s broadcast and begin executing in cycle 6, finishing in cycle 7. Total runtime would be 8 cycles instead of 16. The 9-cycle wait was entirely Mult1’s latency.

Concept Check 6 (Why Renaming Works)

1. Because RS_a is not freed and reallocated until its broadcast completes. By design, an RS’s tag is unique while it is busy. After RS#5 broadcasts and is freed, RS#5 may be allocated to a brand-new instruction — but by then, every earlier instruction that was waiting on the *old* RS#5 has already captured the value. The tag uniqueness invariant is maintained by the “don’t free until broadcast” rule.

Concept Check 7 (Limitations)

1. Example: I1 (add) writes F0. I2 (load) reads memory, takes a long time. I3 (mul) writes F2. If I3 finishes before I1, and then I1 faults (overflow), the architectural state shows F2 has been updated (I3 completed) but F0 has not (I1 faulted). This is “imprecise” because the state is partway through I3 in program order, but I1 (which is earlier) hasn’t completed. The OS handler can’t roll back I3.
2. No, because the execute stage is still out-of-order. Many instructions execute in parallel, their results sit in the ROB, and they wait to commit. The latency-hiding benefit of out-of-order execution is preserved. Commit is fast (one or two instructions per cycle from a buffer of results) — the in-order constraint at commit is cheap, while the in-order constraint at execute (which we’re avoiding) is expensive.